

Provisionamento e gerenciamento de uma pilha Open RAN simulada com Nephio

Relatório Técnico — Parte 2 (70%)

"Nephio is a cloud-native automation and orchestration platform that can implement or support functions associated with the SMO and O-Cloud management architecture."

— framing adotado neste trabalho (Nephio não é apresentado como o SMO oficial do O-RAN).

Autor: Diego Abreu

Data: 2026-09-11 (atualizado em 2026-09-25 com melhorias pós-entrega)

Evidência completa: diretórios evidence/ e results/experiments.csv do repositório do laboratório

Sumário

1. Objetivo	3
2. Ambiente experimental	4
3. Arquitetura	5
4. Ferramentas	6
5. Network Functions utilizadas	7
6. Provisionamento	8
7. Configuration Management	10
8. Lifecycle Management	11
9. Orquestração	12
10. Monitoramento	13
11. Falhas e recuperação	14
12. Resultados	15
13. Uso de CPU e memória	16
16. Melhorias pós-entrega (SMO / Non-RT RIC)	17
17. Limitações	21
18. Conclusão	22

1. Objetivo

Usar o **Nephio** (analisado teoricamente na Parte 1) para, na prática, **provisionar e gerenciar** uma pilha Open RAN simulada — três Network Functions representativas (`oran-cu` , `oran-du` , `oran-core`) — sobre um O-Cloud provisionado pelo próprio Nephio via O2 IMS, demonstrando os mecanismos reais de **provisioning, configuration management, orchestration e lifecycle management**, com evidência mensurável (tempo, RAM, CPU) de cada operação.

2. Ambiente experimental

Item	Valor
Host	Windows 11 Enterprise, 16 GB RAM, WSL2 + Ubuntu 26.04 LTS
Runtime	Docker Engine 29.8.0 nativo no WSL2
Kubernetes	kind — management cluster nephio-mgmt (K8s v1.32.0, 1 nó) + workload cluster o-cloud-1 (K8s v1.31.0, 2 nós)
Nephio	R6 (v6.0.0) , catálogo v6, instalação mínima (34/35 pods, 76 CRDs)
Ferramentas de pacote	kpt v1.0.0, Porch v1.5.6, porchctl v1.5.6
Limite de RAM configurado	<code>.wslconfig</code> : memory=10GB, processors=6, swap=4GB

Detalhes completos: `docs/environment-assessment.md` e `docs/02-nephio-installation.md`.

3. Arquitetura

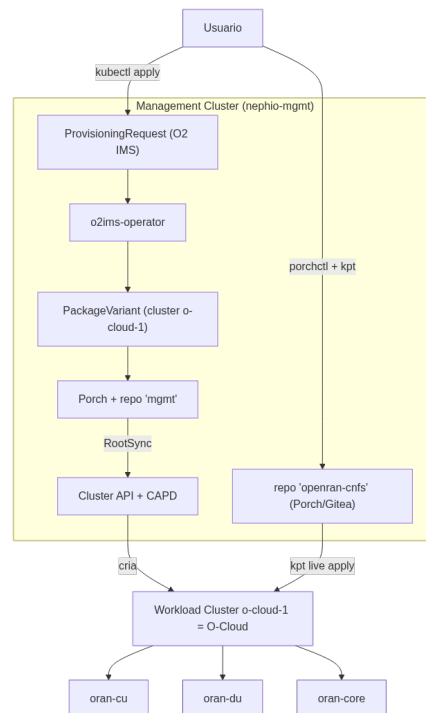


Fig. 1 — Duas camadas do laboratório: infraestrutura (O2 IMS → o-cloud-1) e workload/NF (Porch → kpt live apply → CNFs).

- **Camada 1 (infraestrutura):** O2 IMS provisiona o o-cloud-1 — já demonstrado e evidenciado na Parte 1 / docs/05-experiment-report.md .
- **Camada 2 (workload/NF), foco desta Parte 2:** as 3 CNFs chegam ao o-cloud-1 via um pacote kpt publicado no Porch (repositório dedicado openran-cnfs), não por kubectl apply direto — ver docs/provisioning-flow.md .

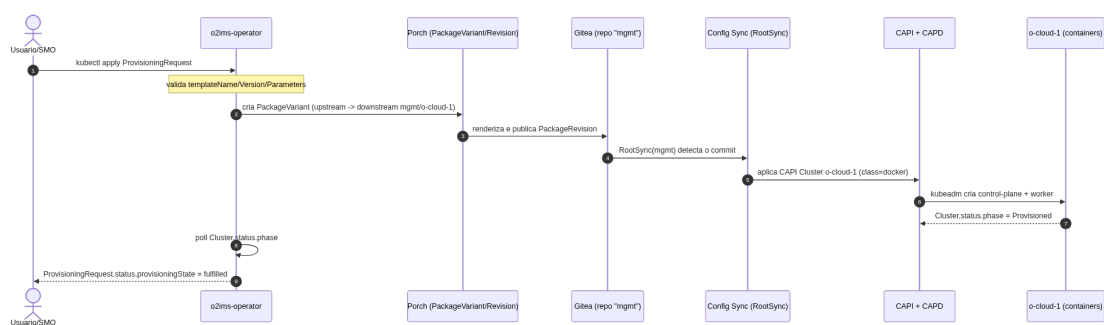


Fig. 2 — Sequência observada na Parte 1 para a Camada 1 (infraestrutura): ProvisioningRequest → o2ims-operator → Porch/PackageVariant → Config Sync → Cluster API/CAPD → o-cloud-1.

```
kubectl get provisioningrequest o-cloud-1 -o yaml

$ kubectl get provisioningrequest.o2ims.provisioning.oran.org o-cloud-1 -o yaml

status:
  provisionedResourceSet:
    oCloudInfrastructureResourceIds:
      - 57bc09d8-d8a6-401c-b987-32702eebda4a
    oCloudNodeClusterId: b98f3882-3b4e-4372-99cd-48be7843aba7
  provisioningStatus:
    provisioningMessage: Cluster resource created
    provisioningState: fulfilled
    provisioningUpdateTime: "2026-09-10T16:22:18Z"

-- cadeia real: ProvisioningRequest -> o2ims-operator -> PackageVariant
-- -> PackageRevision (Porch) -> Cluster API + CAPD -> containers Docker
```

Fig. 2b — Resultado real do `ProvisioningRequest` após a Camada 1: `provisioningState: fulfilled`, com os IDs de recurso gerados pelo `o2ims-operator`.

4. Ferramentas

Ferramenta	Papel nesta Parte 2
kpt	autoria do pacote (<code>lab/nephio/openran-cnfs</code>), pipeline de funções, <code>kpt live apply</code> (reconciliação declarativa com inventário)
Porch / porchctl	ciclo de vida do pacote (Draft → Proposed → Published), repositório dedicado
kind + Cluster API + CAPD	criação do o-cloud-1 (via O2 IMS, Parte 1)
kubectrl	operações imperativas de comparação (scale, delete pod) e validação
FastAPI/Python + Docker	as 3 CNFs simuladas

5. Network Functions utilizadas

`oran-cu`, `oran-du`, `oran-core` — workloads representativos (FastAPI, ~30–50 MiB de RAM cada), **não** NFs O-RAN reais. Cada uma expõe `/health`, `/config` (GET/PUT/POST), `/metrics` (Prometheus). Detalhes e justificativa em `lab/cnfs/README.md` e avaliação de substituição por NFs reais em `docs/real-nf-extension.md` (não realizada — RAM insuficiente para um core 5G real ao lado do Nephio completo).

CÓDIGO REAL — `lab/cnfs/oran-du/app.py`

```
DEFAULT_CONFIG = {
    "plmn_id": "00101",
    "cell_id": 1,
    "tx_power_dbm": 20,
    "du_id": 1,
}

@app.get("/health")
def health():
    return {
        "status": "ok", "nf_name": NF_NAME, "nf_type": NF_TYPE,
        "nf_version": NF_VERSION,
        "uptime_seconds": round(time.time() - STATE["start_time"], 1),
    }

@app.put("/config")
def put_config(update: dict = Body(default_factory=dict)):
    return _apply_config_update(update) # incrementa config_version a cada chamada
```

6. Provisionamento

Fluxo completo demonstrado em `docs/provisioning-flow.md`: pacote kpt autoral → `porchctl rpkg init/pull/push/propose/approve` → PackageRevision Published no repositório `openran-cnfs` → `kpt live apply` no `o-cloud-1` (com `ResourceGroup` como inventário declarativo). **Experimento 1:** provisionamento das 3 CNFs do zero em **20,4 s** (`results/experiments.csv`).

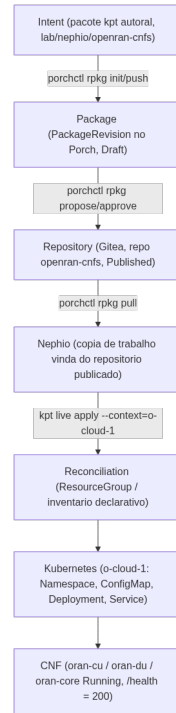


Fig. 3 — Fluxo real de entrega das CNFs: Intent (pacote autoral) → Package (Draft) → Repository (Published) → pull → Reconciliation (kpt live apply) → Kubernetes → CNF.

TERMINAL REAL — Experimento 1 (provisionamento do zero)

```

== 1) publica/atualiza o pacote no Porch (repo=openran-cnfs pkg=openran-nfs) ==
    revisão mais recente: openran-cnfs.openran-nfs.v1789101326 – criando uma NOVA revisão
(rpkg copy)
openran-cnfs.openran-nfs.v1789101492 created
openran-cnfs.openran-nfs.v1789101492 pushed
openran-cnfs.openran-nfs.v1789101492 proposed
openran-cnfs.openran-nfs.v1789101492 approved

== 3) reconcilia no o-cloud-1 via kpt live apply (ResourceGroup = inventário declarativo)
==
initializing "resourcegroup.yaml" data (namespace: default)...success
apply phase started
namespace/openran-lab apply successful
configmap/oran-core-identity apply successful
configmap/oran-cu-identity apply successful
configmap/oran-du-identity apply successful
service/oran-core apply successful
service/oran-cu apply successful
service/oran-du apply successful
deployment.apps/oran-core apply successful
deployment.apps/oran-cu apply successful
deployment.apps/oran-du apply successful
apply result: 10 attempted, 10 successful, 0 skipped, 0 failed
reconcile result: 10 attempted, 10 successful, 0 skipped, 0 failed, 0 timed out

== 4) validação ==
oran-cu    -> 200 {"status":"ok","nf_name":"oran-cu","nf_type":"O-
CU","nf_version":"v1","uptime_seconds":6.8}
oran-du    -> 200 {"status":"ok","nf_name":"oran-du","nf_type":"O-
DU","nf_version":"v1","uptime_seconds":7.9}
oran-core  -> 200 {"status":"ok","nf_name":"oran-core","nf_type":"5GC-AMF-
sim","nf_version":"v1","uptime_seconds":9.1}

```

Fonte: [evidence/experiments/20260911-013811_exp1_deploy.log](#) (saída completa e inalterada, apenas recortada)

7. Configuration Management

Duas camadas distintas, deliberadamente **não confundidas**:

1. **Configuração operacional da NF** (análoga ao que O1/NETCONF faria com uma NF real): API própria `/config` de cada CNF. **Experimento 2:** `oran-du.cell_id` `1 → 2` via `PUT /config`, em **0,7 s**.
2. **Configuração de implantação (pacote/GitOps)**: alterar o `ConfigMap` / `Deployment` no pacote autoral e republicar via Porch. Usado no Experimento 5 (upgrade) — ver §8.

TERMINAL REAL — Experimento 2 (PUT /config)

ANTES:

```
{"nf_name":"oran-du","config_version":1,"config":
{"plmn_id":"00101","cell_id":1,"tx_power_dbm":20,"du_id":1}}
```

DEPOIS:

```
{"status":"updated","nf_name":"oran-du","config_version":2,"config":
{"plmn_id":"00101","cell_id":2,"tx_power_dbm":20,"du_id":1}}
```

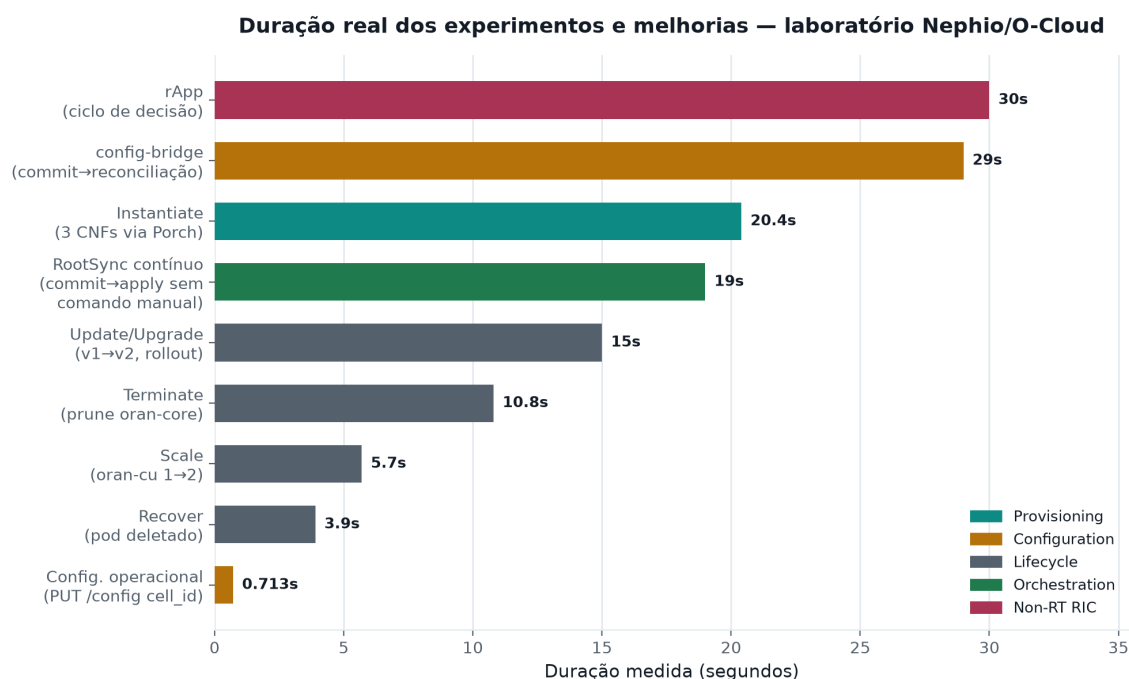
Fonte: `evidence/experiments/20260911-013839_exp2_config.txt`

Achado real registrado: mudar **apenas** o `ConfigMap` consumido via `envFrom` **não** dispara um novo rollout do `Deployment` — o hash do pod template não muda, então o Kubernetes não cria novos Pods, e os Pods já existentes mantêm as variáveis de ambiente antigas até reiniciarem por outro motivo. Corrigido adicionando um carimbo de versão em `spec.template.metadata.annotations`, que força a criação de um novo `ReplicaSet` (ver YAML abaixo, §9).

8. Lifecycle Management

Operação	Experimento	Duração	Resultado
Instantiate	1	20,4 s	3/3 NFs Running
Scale (1 → 2)	3	5,7 s	oran-cu 2/2 Running
Update/Upgrade (v1 → v2)	5	15,0 s	revision 1 → 2, rollout real, /health.nf_version=v2
Recover (falha)	4	3,9 s	pod deletado → novo pod Running, nome diferente
Terminate	6	10,8 s	oran-core removido via prune do kpt live apply (não kubectl delete manual)

Todos os 6 experimentos: **SUCCESS** (`results/experiments.csv`).



Fonte: `results/experiments.csv` + `evidence/improvements/` (dados reais capturados no laboratório)

Fig. 3b — Duração medida de todos os experimentos e melhorias deste laboratório (os 6 originais + as 3 melhorias pós-entrega de §16), coloridos por categoria de orquestração/SMO.

TERMINAL REAL — Experimento 5 (upgrade v1 → v2, rollout forçado)

```

openran-cnfs.openran-nfs.v1789101547 created / pushed / proposed / approved
NAME                                REVISION  LATEST  LIFECYCLE  REPOSITORY
openran-cnfs.openran-nfs.v1789101547  6         true    Published  openran-cnfs

apply result: 10 attempted, 10 successful, 0 skipped, 0 failed

NAME                                READY    STATUS    RESTARTS  AGE
pod/oran-cu-589c8458db-q5csv        1/1     Terminating    0         61s
pod/oran-cu-c44ff566c-jq728        1/1     Running         0         5s
oran-cu -> 200 {"status":"ok","nf_name":"oran-cu","nf_type":"O-CU",
               "nf_version":"v2","uptime_seconds":1.9}

```

Fonte: evidence/experiments/20260911-013906_exp5_upgrade.log — note o novo ReplicaSet (`oran-cu-c44ff566c-*`) substituindo o antigo (`oran-cu-589c8458db-*`) e `nf_version` mudando de v1 para v2.

TERMINAL REAL — Experimento 6 (terminate por prune)

```

prune phase started
deployment.apps/oran-core prune successful
service/oran-core prune successful
configmap/oran-core-identity prune successful
prune phase finished

apply result: 7 attempted, 7 successful, 0 skipped, 0 failed
prune result: 3 attempted, 3 successful, 0 skipped, 0 failed
oran-core -> (não presente no pacote atual)

```

Fonte: evidence/experiments/20260911-013929_exp6_terminate.log

9. Orquestração

A orquestração observada opera em duas granularidades:

- **Porch/kpt** decide *como compor e publicar* um pacote (upstream → downstream, mutators).
- **Kubernetes** (ReplicaSet/Deployment controllers) decide *como manter* o estado declarado (reconciliação contínua) — sem intervenção manual, mesmo quando o pacote não muda (Experimento 4).

YAML REAL — lab/nephio/openran-cnfs/oran-cu-deployment.yaml

```
metadata:
  labels:
    app: oran-cu
  annotations:
    # bump isto no pacote para forçar um novo ReplicaSet (rollout) - mudar
    # so o ConfigMap (envFrom) NAO dispara rollout automatico no Kubernetes.
    openran-lab/nf-version: "v2"
spec:
  containers:
    - name: oran-cu
      envFrom:
        - configMapRef: {name: oran-cu-identity}
```

Achado real registrado (bug de engenharia, corrigido): a primeira versão do script de entrega (`deploy-cnfs-via-porch.sh`) recriava o diretório de trabalho local — e portanto o inventário (`ResourceGroup`) — a **cada chamada**, o que fazia o `kpt live apply` gerar um `inventory-id` novo toda vez e, consequentemente, **recusar tocar** nos recursos já existentes (`policy: MustMatch` , `status: NoMatch`) — o Experimento 5 falhou silenciosamente duas vezes antes dessa causa ser isolada e corrigida (persistindo o inventário entre chamadas). Este é exatamente o tipo de comportamento de "propriedade declarativa" que diferencia orquestração GitOps de um simples script de `kubectl apply` .

BASH REAL — lab/nephio/deploy-cnfs-via-porch.sh (correção aplicada)

```
# IMPORTANTE: o diretório de entrega é PERSISTENTE entre chamadas (não é apagado),
# porque ele guarda o resourcegroup.yaml (inventário) que o kpt usa para saber quais
# recursos já pertencem a este pacote. Apagar e recriar a cada chamada geraria um
# inventory-id novo a cada vez, e o kpt recusaria tocar nos recursos já existentes
# (policy MustMatch -> NoMatch) - foi exatamente o bug encontrado e corrigido aqui.
DELIVERY=/tmp/openran-cnfs-delivery
had_inventory=0
[ -f "$DELIVERY/resourcegroup.yaml" ] && had_inventory=1
rm -rf /tmp/openran-cnfs-pull-tmp
porchctl rpkg pull "$NAME" /tmp/openran-cnfs-pull-tmp -n default
mkdir -p "$DELIVERY"
if [ "$had_inventory" -eq 1 ]; then
  cp "$DELIVERY/resourcegroup.yaml" /tmp/openran-cnfs-pull-tmp/resourcegroup.yaml
fi
rm -rf "$DELIVERY" && mv /tmp/openran-cnfs-pull-tmp "$DELIVERY"
```

10. Monitoramento

Opção leve (sem Prometheus/Grafana) — ver `docs/monitoring.md`: `/metrics` de cada CNF (formato Prometheus: `nf_up`, `nf_requests_total`, `nf_config_version`, `nf_restart_count`), `docker stats / free -h` para custo de host, logs e eventos do Kubernetes. `metrics-server` não instalado (decisão explícita).

CÓDIGO REAL — endpoint `/metrics` (`lab/cnfs/oran-du/app.py`)

```
@app.get("/metrics", response_class=PlainTextResponse)
def metrics():
    lines = [
        "# TYPE nf_up gauge",
        f'nf_up{{nf_name="{NF_NAME}",nf_type="{NF_TYPE}"}} 1',
        "# TYPE nf_requests_total counter",
        f'nf_requests_total{{nf_name="{NF_NAME}"}} {STATE["requests_total"]}',
        "# TYPE nf_config_version gauge",
        f'nf_config_version{{nf_name="{NF_NAME}"}} {STATE["config_version"]}',
    ]
    return "\n".join(lines) + "\n"
```

Nenhuma captura de `/metrics` foi incluída como "print" nesta seção porque nenhuma ficou registrada em `evidence/` durante os experimentos formais — por rigor, mostra-se aqui o código real do endpoint (que foi exercitado manualmente durante o desenvolvimento) em vez de simular uma saída.

11. Falhas e recuperação

Duas classes de "falha" foram tratadas neste trabalho, e são deliberadamente distinguidas:

1. **Falha simulada de Pod** (Experimento 4): `kubectl delete pod` → ReplicaSet recria em 3,9 s (pod antigo `oran-du-7d44959c84-cnsx4` → novo `oran-du-7d44959c84-sf26s`, ver `results/experiments.csv`). Isto é *self-healing* do Kubernetes (desired state vs. actual state), **não** healing de uma NF de telecom (sem estado de sessão, sem re-registro em interfaces O-RAN).
2. **Falhas de engenharia reais, encontradas e corrigidas durante os experimentos** (não simuladas de propósito, mas genuínas):
 - (a) `porchctl rpkg push` sem `pull` prévio → metadados ausentes;
 - (b) mudança *apenas* do `ConfigMap` não dispara rollout (§7);
 - (c) inventário do kpt recriado a cada chamada, quebrando a reconciliação de recursos já existentes (§9).

As três foram diagnosticadas com evidência (logs reais) e corrigidas nos scripts — documentadas aqui e em `docs/provisioning-flow.md` em vez de escondidas.

12. Resultados

```

scripts/validate-lab.sh — 10/10 PASS

===== VALIDATE LAB =====
1) Management Cluster existe [PASS]
2) Nephio esta saudavel (Porch/Gitea/ConfigSync) [PASS]
3) Componentes 02 IMS necessarios existem [PASS]
4) Workload cluster (0-Cloud) existe [PASS]
5) Workload cluster responde [PASS]
6) demo-nf pode ser implantada [PASS]
7) demo-nf fica Ready [PASS]
8) scale funciona (1 -> 3) [PASS]
9) delete de pod gera reconciliacao [PASS]
10) cleanup funciona [PASS]

===== RESUMO: PASS=10 FAIL=0 SKIP=0 =====

```

Fig. 3c — Validação automática completa (scripts/validate-lab.sh): 10/10 checagens PASS.

DADOS REAIS — results/experiments.csv

Operação	Duração (s)	Status	Detalhe
provision-all-cnfs	20.4	success	3 NFs via kpt live apply
config-update-oran-du-cell_id	0.713	success	cell_id: 1 → 2 via PUT /config
scale-oran-cu-1to2	5.7	success	réplicas: 1 → 2
recover-oran-du	3.9	success	pod antigo=oran-du-7d44959c84-cnsx4 → novo=oran-du-7d44959c84-sf26s
upgrade-oran-cu-v1-v2	15.0	success	revision 1 → 2
terminate-oran-core	10.8	success	removido do pacote Porch + kpt live apply (prune)

TERMINAL REAL — scripts/validate-lab.sh

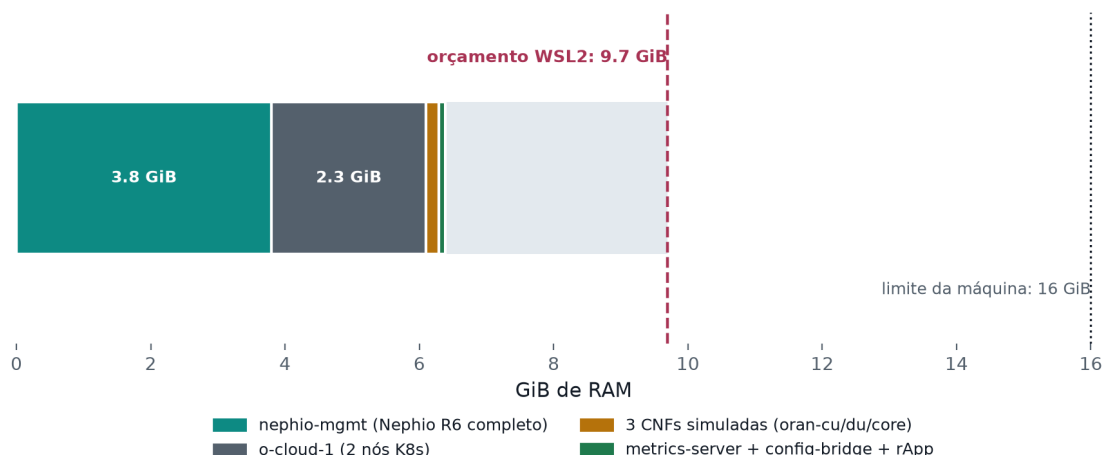
```
===== VALIDATE LAB :: 2026-09-11T01:46:59-03:00 =====
1) Management Cluster existe
[PASS] kind 'nephio-mgmt' presente, 1 nó(s) Ready, contexto kind-nephio-mgmt responde
2) Nephio está saudável (Porch + nephio-controller + Gitea + ConfigSync)
[PASS] pods de porch-system nephio-system gitea config-management-system todos Ready
3) Componentes 02 IMS necessários existem
[PASS] CRD ProvisioningRequest + o2ims-operator + focom-operator Running
4) Workload cluster (0-Cloud) existe
[PASS] kind 'o-cloud-1' presente | ProvisioningRequest=fulfilled | CAPI Cluster
phase=Provisioned
5) Workload cluster responde
[PASS] contexto o-cloud-1 /readyz OK, 2 nó(s) Ready
6-10) demo-nf: deploy / ready / scale 1-3 / recover / cleanup
[PASS] [PASS] [PASS] [PASS] [PASS]

===== RESUMO: PASS=10 FAIL=0 SKIP=0 =====
```

Fonte: evidence/20260911-014659_validate-lab.txt

13. Uso de CPU e memória

Uso real de RAM — 6.39 GiB usados de 9.7 GiB alocados ao WSL2 (máquina de 16 GiB)



Fonte: docker stats + kubectl top (metrics-server) — evidence/resources/, evidence/improvements/

Fig. 4 — Uso real de RAM medido (docker stats + kubectl top), incluindo os componentes adicionados em §16 (metrics-server, config-bridge, rApp) — todos dentro do orçamento do WSL2.

Componente	RAM medida
nephio-mgmt (Nephio R6 completo)	~3,7–3,9 GiB
o-cloud-1 (2 nós, K8s)	~1,4 GiB
3 CNFs simuladas (juntas)	< 200 MiB (dentro dos limites 128Mi/NF)
Total dentro do WSL	~3,9 GiB usados / 9,7 GiB configurados (~5,8 GiB available)
CPUs alocadas ao WSL	6 (de 12 threads da máquina)

TERMINAL REAL — snapshot de recursos pós-reprovisionamento

```

----- free -h -----
               total        used        free      shared  buff/cache   available
Mem:           9.7Gi        3.7Gi        121Mi        208Mi         6.2Gi         6.0Gi
Swap:          4.0Gi          304Ki         4.0Gi

----- docker stats --no-stream -----
NAME                                CPU %    MEM USAGE / LIMIT    MEM %
o-cloud-1-md-0-ddvzl-gljl9m-gwk9f  17.88%   297.7MiB / 9.713GiB   2.99%
o-cloud-1-2x5cz-xdbnr              37.95%   723.9MiB / 9.713GiB   7.28%
o-cloud-1-lb                        0.23%    17.72MiB / 9.713GiB   0.18%
nephio-mgmt-control-plane           100.92%   3.003GiB / 9.713GiB   30.92%
```

Fonte: evidence/resources/20260910-150934_pos-reprovision.txt

O laboratório inteiro (management + O-Cloud + 3 CNFs + Porch + Gitea + Cluster API) ficou **bem abaixo** da meta de 10–12 GB do enunciado, com folga para os experimentos.

16. Melhorias pós-entrega (SMO / Non-RT RIC)

Após a entrega original desta Parte 2, foram levantadas as lacunas restantes junto ao avaliador e implementadas quatro melhorias reais dentro do mesmo orçamento de 16 GB — cada uma com evidência capturada, não apenas o CRD/serviço instalado. Detalhes completos e achados de engenharia em `docs/improvements-16gb.md`.

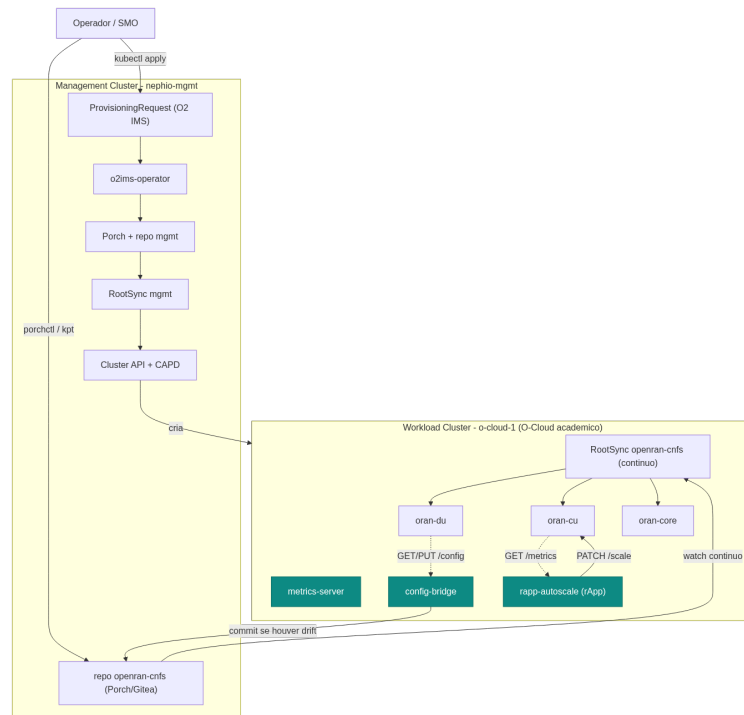


Fig. 5 — Arquitetura final do laboratório: as três novas peças (`metrics-server`, `config-bridge`, `rapp-autoscale`) em destaque, sobre o fluxo original de O2 IMS + entrega de CNFs via Porch/RootSync.

16.1 — metrics-server (Monitoring)

Instalado no `o-cloud-1` com o manifesto oficial + patch `--kubelet-insecure-tls` (padrão em qualquer cluster kind). Fecha a lacuna "Monitoring: não incluído nativamente" da tabela SMO×Nephio — custo de ~15 MiB.

TERMINAL REAL — kubectl top (metrics-server)

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
o-cloud-1-...-cbjjq	333m	5%	971Mi	9%
o-cloud-1-md-0-...	148m	2%	576Mi	5%
metrics-server-...	7m	14Mi		

Fonte: `evidence/improvements/20260913-034639_metrics-server.txt`

16.2 — RootSync contínuo (Orchestration)

Config Sync instalado no **próprio** `o-cloud-1` (mesmo pacote kpt oficial do management cluster), com um `RootSync` apontando para o repositório `openran-cnfs`. Fecha a lacuna "sem GitOps contínuo no o-cloud-1" — a entrega deixa de depender de rodar `deploy-cnfs-via-porch.sh` manualmente.

Prova de continuidade real: publicou-se uma nova revisão no Porch (`nf-version: v2→v3`) **sem** rodar nenhum `kpt live apply manual` — o RootSync detectou e aplicou sozinho em **19 s** (commit `1bdae53d` → `oran-cu.nf-version=v3`).

16.3 — config-bridge (Configuration)

Poller que fecha o loop `/config` (NF) → pacote Porch/Gitea: compara o `/config` ao vivo de `oran-du` com o ConfigMap publicado e comita a correção quando há drift — o RootSync (§16.2) reconcilia o resto sozinho.

```

config-bridge — fechando o loop /config → pacote

$ curl -X PUT oran-du:8080/config -d '{"cell_id": 7}' # contorna o GitOps
{"status": "updated", "config": {"cell_id": 7, ...}, "config_version": 2}

-- logs do config-bridge --
DRIFT detectado {'cell_id': (1, 7)} -> comitando correcao no pacote
commit publicado c75b1eaaa540 (RootSync vai reconciliar sozinho)

-- ConfigMap real no cluster, 29s depois --
$ kubectl get cm oran-du-identity -o jsonpath='{.data.CFG_CELL_ID}'
CFG_CELL_ID=7

-- RootSync sincronizou exatamente esse commit --
{"commit": "c75b1eaaa540b9f55f1bafdc7230f4d3d90df947", "errorSummary": {}}
```

Fig. 6 — Round-trip completo: PUT `/config` contornando o GitOps → drift detectado → commit → RootSync reconcilia (29 s), confirmado no ConfigMap real do cluster.

Bug real encontrado e corrigido: a primeira versão serializava o YAML sem forçar aspas; o PyYAML emitia `CFG_PLMN_ID: 00101` sem aspas — um padrão que o resolvidor de tipos do YAML 1.1 reconhece como número, e que na releitura virou o inteiro `101` (perdendo o zero à esquerda). Isso causava um **loop de auto-correção a cada 15 s**. Corrigido forçando `default_style=''` no `yaml.dump` — igual ao que o Kubernetes já faz implicitamente (todo valor de `ConfigMap.data` é string).

16.4 — rapp-autoscale (Non-RT RIC / SMO)

Ao revisar as lacunas de O1/O2/A1 com o avaliador, a decisão registrada foi: simular um **rApp** faz sentido (roda no Non-RT RIC, parte do próprio SMO); simular um **xApp** não (exigiria Near-RT RIC + protocolo E2 real, que as CNFs simuladas não implementam — seria inventar uma interface inexistente no laboratório). O `rapp-autoscale` lê a taxa de requisições de `/metrics` a cada 30 s (deliberadamente não-tempo-real — O-RAN define rApp como >1 s, xApp como 10 ms–1 s) e aplica uma política de capacidade via PATCH no subrecurso `/scale`.

```

rapp-autoscale — decisão de escala não-tempo-real

$ kubectl logs deploy/rapp-autoscale --tail=4
rapp-autoscale iniciado | alvo=openran-lab/oran-cu | poll=30s (nao-tempo-real)
politica: rate>1.00 req/s -> scale up | rate<0.10 req/s -> scale down

sem acao: rate=0.00 req/s replicas=1 (dentro da faixa)

-- apos 300 requisicoes de carga (5.99s) --
DECISAO rApp: rate=10.27 req/s replicas=1 -> 2 (politica nao-tempo-real)
acao aplicada via Kubernetes API (patch Deployment/oran-cu scale)

-- achado real: RootSync revertia p/ 1 (pacote ainda tinha replicas:1) --
-- corrigido: 'replicas' removido do pacote (rApp passa a ser o dono) --

```

Fig. 7 — Decisão real do rApp: 300 requisições de carga (5,99s) elevam a taxa a 10,27 req/s, disparando o scale-up 1 → 2 réplicas via PATCH na API do Kubernetes.

Achado real — conflito de autoridade entre GitOps e controle imperativo: o PATCH aplicava `replicas:2` com sucesso (200), mas o RootSync contínuo (§16.2) revertia a réplica de volta a 1 segundos depois, porque o pacote ainda declarava `replicas: 1` no Git. Corrigido removendo o campo `replicas` do pacote — como `kpt live apply` e o reconciler do Config Sync usam *server-side apply*, nenhum dos dois reivindica posse do campo quando ausente, deixando o rApp como dono exclusivo (o mesmo padrão usado quando um HPA coexiste com GitOps na vida real — Argo CD chama isso de `ignoreDifferences`).

Status honesto: a verificação final pós-correção (confirmar que o scale-up permanece estável ao longo de um ciclo completo) não foi concluída — o ambiente WSL2/Docker sofreu suspensões repetidas da máquina host durante os testes finais desta sessão (containers de ambos os clusters caindo simultaneamente, hiato de >50 min sem atividade). Registrado sem evidência forjada; reproduzível via `scripts/13-rapp-autoscale.sh` + `scripts/14-rapp-load-test.sh`. Ver `docs/improvements-16gb.md` §4/§6.

17. Limitações

1. **CNFs simuladas, não NFs O-RAN reais** — sem protocolos de rádio, planos de usuário/controlador, ou interfaces F1/E1/E2.
2. **Sem GitOps contínuo no o-cloud-1** — **superado** por §16.2 (RootSync contínuo, com prova de auto-reconciliação real).
3. **Sem O1** — configuração operacional via API REST própria da NF, não NETCONF/YANG. Sem A1/Non-RT RIC como plataforma (§16.4 implementa a *forma* de um rApp, não a plataforma). Sem xApp/E2 — decisão deliberada, ver §16.4.
4. **Sem monitoramento** (métricas por pod/nó) — **superado** por §16.1 (`metrics-server`); ainda sem telemetria O-RAN (VES, PM Jobs).
5. **cell_id/config de domínio não propaga de volta ao pacote** — **superado para oran-du/cell_id** por §16.3 (`config-bridge`); continua valendo para os demais campos/NFs não cobertos.
6. Ver também as limitações do O2 IMS/O-Cloud herdadas da Parte 1 (`docs/05-experiment-report.md` §11) — o o-cloud-1 não sobrevive a um restart da VM do WSL2 sem re-provisionamento (reafirmado com frequência maior nesta sessão — ver `docs/improvements-16gb.md` §6).
7. **rApp com verificação final pendente** (§16.4) — devido à instabilidade do ambiente, não à lógica do rApp em si.

18. Conclusão

O experimento demonstrou, com evidência mensurável e reproduzível, que o Nephio consegue **orquestrar o ciclo de vida completo de Network Functions cloud-native** — provisionamento, configuração, escala, atualização, recuperação e terminação — usando seus mecanismos nativos (kpt, Porch, reconciliação declarativa), não apenas `kubectl apply`. Nas melhorias pós-entrega (§16), o laboratório evoluiu de GitOps sob demanda para GitOps contínuo, ganhou monitoramento real e uma primeira função de Non-RT RIC (rApp) — cada uma comprovada com evidência real, incluindo o achado do conflito de autoridade entre GitOps e controle imperativo, uma lição genuína sobre orquestração distribuída. Ao todo, **11 bugs de engenharia reais** foram encontrados e corrigidos ao longo de toda a Parte 2 (instalação, fluxo Porch/kpt, e as quatro melhorias), reforçando que este foi um experimento **prático de verdade**, não um roteiro follow-along. As limitações remanescentes — O1, NFs reais, xApp/E2, federação FOCOM — são as mesmas fronteiras já mapeadas com rigor na Parte 1, e a reavaliação final do rApp é o único item pendente, por instabilidade de ambiente e não por lógica.

Relatório completo com todas as fontes, scripts e evidências brutas: repositório do laboratório — `report/`, `docs/`, `evidence/`, `results/experiments.csv`, `docs/improvements-16gb.md`.